

iOS 项目工具类与第三方库

iOS 项目工具类与第三方库详解

一、ZYUtils - 通用功能百宝箱

ZYUtils 是项目中的核心工具类，集成多种通用功能，类似“瑞士军刀”，覆盖“进度显示、设备信息、数据存储、工具方法”等高频开发场景，减少重复编码。

1. 进度显示功能

基于 MBProgressHUD 实现，用于耗时操作（如网络请求、数据处理）时显示加载指示器，提升用户等待体验。

核心代码：

```
// 1. 简化版：显示进度提示框（默认垂直偏移 -30）
+ (void)showProgressView:(NSString *)strText
view:(UIView *)view
modal:(BOOL)bModal {
    [ZYUtils showProgressView:strText view:view offset:-30 modal:bModal];
}

// 2. 完整版：支持自定义垂直偏移，控制背景是否可点击
+ (void)showProgressView:(NSString *)strText
view:(UIView *)view
offset:(NSInteger)nOffset
modal:(BOOL)bModal {
    // 创建 MBProgressHUD 实例并添加到目标视图
    MBProgressHUD *HUD = [MBProgressHUD showHUDAddedTo:view animated:YES];
    HUD.userInteractionEnabled = !bModal; // modal=YES 时禁止背景点击（防止误操作）
    HUD.labelText = strText; // 设置提示文本（如“加载中...”）
    HUD.yOffset = nOffset; // 调整垂直位置（避免遮挡关键内容）
}

// 3. 隐藏进度提示框
+ (void)hideProgressView:(UIView *)view {
```

```
[MBProgressHUD hideHUDForView:view animated:YES];
}
```

使用场景：网络数据加载、文件导出、复杂计算等耗时操作，告知用户“操作正在进行中”，避免用户误以为应用卡顿。

2. 设备信息获取功能

提供设备型号、系统版本等关键信息的获取方法，支持根据设备特性适配功能（如不同系统版本、设备型号的差异化逻辑）。

核心代码：

```
// 1. 获取设备型号（如“iPhone 13 Pro”“iPad Pro 12.9-inch (5th generation)”）
NSString *osDevice() {
    NSString *platform = [[UIDevice currentDevice] platformString];
    return platform;
}

// 2. 获取系统版本字符串（如“16.5”“17.0”）
NSString *osVersion() {
    return [[UIDevice currentDevice] systemVersion];
}

// 3. 系统版本转为整数（便于版本比较，如“14.2”→140200、“16.1.1”→160101）
NSInteger osVersionInteger() {
    NSString *version = [[UIDevice currentDevice] systemVersion];
    NSArray *versionSegments = [version componentsSeparatedByString:@"."];
    NSInteger versionValue = 0;
    NSInteger multiplier = 1000000; // 乘数：从高版本位到低版本位（百万、万、百）
    for (NSString *segment in versionSegments) {
        versionValue += [segment integerValue] * multiplier;
        multiplier /= 100; // 每段版本号占两位（如“14.2”中“14”占两位、“2”补为“02”）
    }
    return versionValue;
}
```

使用场景：

- 系统版本适配（如 iOS 13+ 支持暗黑模式、iOS 15+ 支持 `UINavigationController` 新特性）；
- 设备型号适配（如 iPad 横屏布局、iPhone SE 小屏适配）。

3. UserDefaults 数据存取

封装 `NSUserDefaults` 操作，简化用户偏好设置、应用状态等轻量数据的“读、写、删”流程，避免重复调用系统 API。

核心代码：

```
// 1. 读取数据（根据 key 获取存储的对象，支持 NSString、NSNumber、NSArray 等可归档类型）
id getUserDefaultValue(NSString *key) {
    return [[NSUserDefaults standardUserDefaults] objectForKey:key];
}

// 2. 保存数据（将对象存储到 UserDefaults，并立即同步到磁盘）
void saveUserDefaultValue(id object, NSString *key) {
    NSUserDefaults *userDefaults = [NSUserDefaults standardUserDefaults];
    [userDefaults setObject:object forKey:key];
    [userDefaults synchronize]; // iOS 10+ 可省略（系统自动异步同步），低版本需保留
}

// 3. 删除数据（根据 key 移除存储的对象）
void removeUserDefaultValue(NSString *key) {
    NSUserDefaults *userDefaults = [NSUserDefaults standardUserDefaults];
    [userDefaults removeObjectForKey:key];
}
```

使用场景：保存用户设置（如“是否开启推送通知”“默认主题色”）、应用状态（如“上次登录账号”“最后浏览页面”）等轻量数据（不建议存储大体积数据或敏感信息）。

4. 其他常用功能

ZYUtils 还集成以下实用工具，覆盖日常开发多场景需求：

功能分类	核心方法示例	用途说明
日期时间处理	<code>+ (NSString *)getCurrentTimeString:(NSString *)format</code>	获取指定格式的当前时间（如“yyyy-MM-dd HH:mm:ss”“MM/dd”）
颜色处理	<code>+ (UIColor *)getColor:(NSString *)hexString</code>	从十六进制字符串（如“#FF0000”“0x00FF00”）创建 UIColor 对象

	<code>+ (UIColor *)randomColor</code>	生成随机颜色（用于测试、标签分类等场景）
图片处理	<code>+ (UIImage *) getImageWithColor:(UIColor *)color</code>	生成纯色图片（用于按钮背景、占位图等）
	<code>+ (UIImage *) imageWithMaxSide:(CGFloat)maxSide image:(UIImage *)image</code>	按最大边长缩放图片（避免图片过大导致内存占用过高）
弹窗提示	<code>+ (void)showAlert:(NSString *)message</code>	简化警告弹窗创建（替代UIAlertController 重复代码）
	<code>+ (void)CCAlertWithTitle: (NSString *)title message:(NSString *) message</code>	带标题的弹窗提示
网络连接检查	<code>+ (BOOL) connectedToNetwork</code>	判断设备是否联网（基于Reachability 框架，支持 WiFi / 蜂窝网络）
视图控制	<code>+ (void)enableView:(UIView *)view enabled:(BOOL)enabled</code>	批量启用 / 禁用视图及子视图交互（如按钮置灰、输入框禁用）

二、NSObject+ZYAddtion - NSObject 的扩展

通过分类为所有 NSObject 子类添加通用方法，无需继承即可使用，重点增强 “URL 处理” 和 “类信息获取” 能力，提升代码复用性。

1. URL 查询参数处理

提供 URL 解析与参数提取功能，支持从带参数的 URL（如 `https://example.com/page?name=test&age=18&desc=ios+dev`）中快速获取查询参数，避免手动分割字符串的繁琐操作。

核心代码：

```
// 1. 从 URL 查询参数列表中获取指定 key 的值（适用于已解析的 queryItems）
- (NSString *)zyValueForKey:(NSString *)key
```

```

fromQueryItems:(NSArray<NSURLQueryItem *> *)queryItems {
// 使用谓词过滤出 name 与 key 匹配的 queryItem
NSPredicate *predicate = [NSPredicate predicateWithFormat:@"name == %@", key];
NSURLQueryItem *targetItem = [[queryItems filteredArrayUsingPredicate:predicate]
firstObject];
return targetItem.value; // 返回参数值（自动处理 URL 编码，如 “ios+dev” → “ios dev” ）
}
// 2. 解析 URL 字符串，提取所有查询参数（含原始 URL、URL 头部）
- (NSMutableDictionary *)zyGetDataOfQueryString:(NSString *)url {
// 校验 URL 合法性
if (!url || url.length == 0) {
NSLog(@"error! url is nil or empty");
return nil;
}
// 分离 URL 头部（“?” 之前）与查询部分（“?” 之后）
NSArray *urlSegments = [url componentsSeparatedByString:@"?"];
NSMutableDictionary *resultDict = [[NSMutableDictionary alloc] init];
[resultDict setObject:url forKey:@"url"]; // 保存原始 URL
// 无查询参数时返回（仅含原始 URL 和头部）
if (urlSegments.count < 2) {
NSLog(@"error! url has no query parameters: %@", url);
[resultDict setObject:urlSegments[0] forKey:@"header"];
return resultDict;
}
// 保存 URL 头部（如 “https://example.com/page” ）
[resultDict setObject:urlSegments[0] forKey:@"header"];
// 解析查询参数（以 “&” 分割每个参数，如 “name=test&age=18” → ["name=test", "age=18"]）
NSArray *paramSegments = [[urlSegments[1] stringByRemovingPercentEncoding]
componentsSeparatedByString:@"&"];
for (NSString *paramSegment in paramSegments) {
// 分离参数名与参数值（以 “=” 分割，处理参数值含 “=” 的场景，如 “desc=ios=dev” ）
NSArray *keyValueSegments = [paramSegment componentsSeparatedByString:@"="];
if (keyValueSegments.count >= 2) {
// 拼接参数值（处理参数值含 “=” 的情况）
NSString *paramKey = keyValueSegments[0];
NSString *paramValue = @"";
for (NSInteger i = 1; i < keyValueSegments.count; i++) {
if (i > 1) {
paramValue = [paramValue stringByAppendingString:@"="]; // 补全 “=”
}
}
}
}
}

```

```

paramValue = [paramValue stringByAppendingString:keyValueSegments[i]];
}
[resultDict setObject:paramValue forKey:paramKey];
} else if (keyValueSegments.count == 1) {
// 仅参数名无值（如“debug”），值设为空字符串
[resultDict setObject:@"" forKey:keyValueSegments[0]];
}
}
return resultDict;
}

```

使用场景：

- 处理 Deeplink 跳转参数（如从短信 / 网页点击链接打开应用，提取跳转目标页面、用户 ID 等参数）；
- 解析网络请求回调 URL 中的参数（如第三方登录回调、支付结果回调）。

2. 类名获取

提供简洁方法获取类名（自动去除命名空间前缀），便于日志打印、反射编程或动态创建类实例。

核心代码：

```

// 获取类名（去除命名空间前缀，如“DemoApp.UserModel” → “UserModel”）
+ (NSString *)className {
    NSString *fullClassName = NSStringFromClass(self);
    NSArray *classNameSegments = [fullClassName componentsSeparatedByString:@"."];
    return classNameSegments.lastObject; // 返回最后一段（即纯类名）
}

```

使用场景：

- 日志记录（如 “[UserModel] 数据解析失败”，清晰标识类来源）；
- 反射编程（如根据类名动态创建实例：`Class modelClass = NSClassFromString([UserModel className]); id model = [[modelClass alloc] init];`）；
- 单元格复用标识（如 `[UserCell className]` 作为 UITableViewCell 重用标识，避免硬编码）。

三、Third 文件夹内容

Third 文件夹存放项目依赖的第三方库，覆盖 “数据解析、内购、社交、统计、多媒体” 等核心功能，避免 “重复造轮子”，降低开发成本。

1. 核心第三方库说明

库名称	功能说明	项目中的应用场景
JSONKit	高性能 JSON 解析与序列化库（支持 NSString/NSData 与 NSDictionary/NSArray 互转）	网络请求返回 JSON 数据的解析（如接口响应数据转为字典/数组）
MKStoreKit	应用内购买（IAP）流程封装（处理产品请求、交易验证、购买记录保存）	处理 “移除广告” “解锁高级功能” 等内购产品的购买逻辑
iRate	App Store 评分提示工具（支持设置使用天数、使用次数后触发提示）	满足 “用户使用 5 天且打开 10 次” 条件后，提示用户前往 App Store 评分
libWeixin	微信开放平台 SDK 封装（支持微信分享、登录、支付）	微信分享内容（如文章、图片）、微信授权登录、微信支付
MWPhotoBrowser	图片浏览组件（支持多图预览、手势缩放、保存到相册、长按操作）	相册图片预览、文章配图查看、头像大图查看等场景
UMAnalytics_Sdk_3.6.7	友盟统计 SDK（支持页面访问、事件点击、用户留存等数据统计）	统计应用活跃度、关键功能点击量（如 “广告点击” “内购按钮点击”）
WHC_ModelSqliteKit	数据库操作工具（支持 Model 与 SQLite 数据库直接映射）	本地数据持久化存储（如离线缓存用户列表、历史记录）
YTAudioRecorderController	音频录制与播放控制器（支持录音、暂停、播放、音频格式转换）	语音评论、语音消息录制与播放功能
AdmobSDK	Google 广告 SDK（支持横幅广告、插屏广告、视频广告）	应用内展示广告（如首页横幅广告、游戏关卡结束插屏广告）

四、总结

1. **ZYUtils** 作为通用工具类，整合进度显示、设备信息、数据存储等高频功能，减少重复代码，提升开发效率，是项目中的“基础工具集”；
2. **NSObject+ZYAddtion** 通过分类扩展增强所有对象的能力，尤其 URL 参数解析功能在网络场景中实用价值高，无需继承即可复用；
3. **Third 文件夹** 中的第三方库覆盖多核心功能，避免重复开发，但部分库版本较旧（如 JSONKit、iRate），现代 iOS 开发中可考虑更新替代方案（如系统 `NSJSONSerialization` 替代 JSONKit、`SKStoreReviewController` 替代 iRate）。

这些工具类与第三方库的使用，使项目代码更模块化、易维护，同时降低了开发成本，是 iOS 项目开发中“高效编码”的典型实践。

（注：文档部分内容可能由 AI 生成）